

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 06

বক্তৃতা – 27

কীভাবে লকিং করে (হার্ডওয়্যার সমাধান ও তার ব্যবহার)

How to implement Locking (Hardware Solutions and Usage)

হ্যালো। এই বিশেষ ভিডিওতে আমরা critical section problem সমাধানের জন্য হার্ডওয়্যার সলিউশন দেখব। অতএব আগের ভিডিওতে আমরা critical section problem সমাধানে software solution দেখেছি। যাইহোক, বাস্তবে এই সমাধান খুব দক্ষ হয় না। সুতরাং বাস্তবে উভয়ই অপারেটিং সিস্টেমের পাশাপাশি অ্যাপ্লিকেশনেও, হার্ডওয়্যার কৌশল দক্ষতার সাথে critical section সমস্যার সমাধান করার জন্য ব্যবহার করা হয়। তাই আমরা ছোট motivating উদাহরণের সঙ্গে এই ভিডিওটি শুরু করব।

(স্লাইড সময় : 00:52 দেখুন)

সুতরাং আসুন আমরা এই বিশেষ উদাহরণ দিয়ে শুরু করি। ধরা যাক দুটি প্রসেস আছে; প্রক্রিয়া 1 এবং প্রক্রিয়া 2 এবং উভয় একটি অনুরূপ critical section আছে যা একই shared data access করে। আরো মনে করি এই shared variable lock আছে, তাই এই variable প্রক্রিয়া 1 এবং প্রক্রিয়া 2 এর মধ্যে share করা হয়। এখন এই প্রতিটি প্রক্রিয়ায় দুটি জিনিস হয়; প্রথমে critical section lock করার জন্য, এই lock টি 0 না হওয়ার আগে পর্যন্ত while লুপটি প্রথমে execute করতে থাকবে। লকটি 0 হয়ে গেলে প্রক্রিয়াটি এখানে প্রবেশ করবে এবং লকটিকে 1 এ সেট করবে এবং তারপর critical section চালানো হবে। Critical section শেষে প্রক্রিয়াটি লক 0 তে সেট করবে যাতে বিভাগটি আনলক হয়। যে প্রক্রিয়াটি আমি এখানে করবো তা হল; এই বিশেষ স্কিম mutual

exclusion অর্জন করে? উত্তর হল না। মূলত আমরা পূর্বের ভিডিওতে এমন জিনিসগুলি দেখেছি। উভয় প্রসেসের মধ্যে context switch করার কারণে আমরা এক সময় একটি state এ পৌঁছাবো যখন উভয় প্রসেসগুলি critical section এ execute হবে। সুতরাং, mutual exclusion অর্জন করতে সক্ষম হবে না। সুতরাং, উদাহরণস্বরূপ, যদি আমরা দেখি, আমরা দেখব যে লকের প্রাথমিক মান 0। তারপর ধরি প্রসেস P1 CPU এর মধ্যে সঞ্চারিত হয় আর এটা while লকে execute হয় যেটা 0 নয় এবং যেহেতু, এখানে 0 হিসাবে লক আছে, তাই এই নির্দিষ্ট while লুপটি ভাঙা হবে, তবে একটি context switch হয় যা প্রক্রিয়াটি লক 1 এর সেট করার আগে। এই context switch এর ফলে প্রসেস P2 execute করবে। এখন P2 ও লকটির মান 0 দেখতে পাবে, এবং লুপ থেকে প্রস্থান করে লককে 1 এ সেট করবে। এখন আবার যদি কোন context switch ঘটে থাকে এবং P1 প্রক্রিয়াটি চালু করা হয় এটি যেখানে বন্ধ করে দেওয়া হয়েছিল সেখান থেকে, এটা লকটির পুরানো মান দেখবে 0 কারণ, context switch ঘটে যাওয়ার আগে প্রক্রিয়াটির পুরাতন মান ফিরিয়ে দেওয়া হয়। এবং অতএব, P1 প্রক্রিয়াটি লকটির মানকে 0 হিসাবে দেখতে পাবে। এবং তাই, এটি 1 এর সমান লক সেট করবে এবং critical section অংশে প্রবেশ করবে। এইভাবে আমরা এমন state পৌঁছেছি যেখানে উভয় প্রসেসগুলি critical section এ রয়েছে।

(স্লাইড সময় 03:42 দেখুন)

এখন, এই বিশেষ স্কিমটি ব্যর্থ হওয়ার মূল কারণটি ছিল যে আমরা সফটওয়্যারে এইটি লিখেছি, আমরা এই দুটি statement atomic করতে সক্ষম ছিলাম না। যে, এই দুটি statement এর মধ্যে একটি context switch ঘটে পারে না তা আমরা নিশ্চিত করতে অক্ষম। আরেকটি উপায় বলে, আমরা এই দুটি statement একক ইউনিট হিসাবে চালানো করতে অক্ষম। এবং এইভাবে, এই কারণে আমরা mutual exclusion অর্জন করতে পারিনি। এখন বেশিরভাগ প্রসেসর যেমন x86 ইন্টেল প্রসেসরগুলি নির্দেশাবলী নিবেদিত করেছে যা নিশ্চিত করে যে এই statement atomic ভাবে সঞ্চারিত হবে। আসুন আমরা এমন একটি atomic নির্দেশাবলীর উদাহরণ দেখি যার মাধ্যমে আমরা এই বিধি একটি atomic পদ্ধতিতে প্রয়োগ করতে পারি। তাই আমরা প্রথমে একটি খুব সাধারণ নির্দেশনা গ্রহণ করব এবং তারপর ইন্টেল x86 প্রকারের প্রসেসরের জন্য বিশেষভাবে বিশ্লেষণ করব।

(স্লাইড সময় 04:51 দেখুন)

আমরা আজ যে প্রথম instruction দেখি তা হল Test and Set instruction। মূলত যদি আমরা C তে instruction লিখি এটির এই মত কিছু চেহারা হবে। এটি একটি function যা একটি মেমরি অবস্থানের একটি pointer নেয়। সুতরাং এই function প্রথম যে মেমরি অবস্থানের বিষয়বস্তু previous নামক variable এর মধ্যে সংরক্ষিত হয়, তারপর সেই মেমরি 1 এ সেট করা হয় এবং return value পূর্ববর্তী মান হয়। অন্য কথায়, এই function ফাংশনটি মেমরির পূর্ববর্তী মান ফিরিয়ে দেবে এবং মেমরিটিতে 1 এর মান সেট করবে। এখন, যখন আমরা এই হার্ডওয়্যার প্রেক্ষাপটে এবং মূলত প্রসেসরের দৃষ্টিকোণ থেকে দেখি, এই পুরো function একটি

atomic function হয়। মূলত আমাদের একটি নির্দেশ আছে যেটা এই জিনিসটি এক শটের মধ্যে হবে, আমাদের একটি নির্দেশ আছে যে এই অপারেশন সব atomically সঞ্চালন হবে। সুতরাং আসুন আমরা এই নির্দিষ্ট চিত্রটি দেখি যে এটি আসলে কীভাবে কাজ করে। তাই আসুন আমরা বলি আমাদের প্রসেসর আছে এবং মেমরি অবস্থান L দ্বারা নির্দেশিত এখানে আছে। সুতরাং, এই Test and Set সঞ্চালিত হবে কি হবে, প্রথমে মেমরির পূর্ববর্তী content গুলি প্রসেসরটিতে উপস্থিত একটি রেজিস্টারে লোড হয়ে যায় এবং তারপর মেমরি সামগ্রী 1 এ সেট করা হয়। সুতরাং আসুন, এটি আরেকটি সময় দেখি; প্রথমে মেমরির content L দ্বারা নির্দেশিত হয় যা প্রসেসরটিতে উপস্থিত একটি register এ লোড হয় এবং তারপর সেই মেমরি অবস্থার content 1 এ সেট করা হয়।

(স্লাইড সময় 06:46 দেখুন)

এখন, Test and Set instruction এর atomocity এই statement এখানে সংজ্ঞায়িত বা ব্যাখ্যা করা হয়েছে যা এই, যদি দুটি সিপিইউ Test and Set পরীক্ষা চালায় ঠিক একই সময়ে তবে একটি প্রসেসর নিশ্চিত করে যে এই প্রসেসরগুলির মধ্যে একটির জন্য উভয় Test and Set করা অপারেশন হয়। মেমরির একটি content পড়া এবং নির্দিষ্ট মেমরি অবস্থান সেট করা, তাই এই উভয় পদক্ষেপগুলি সম্পন্ন হয় অন্য প্রক্রিয়া শুরু করার আগে। অন্য কথায়, দুটি প্রসেসরের জন্য পরীক্ষা চালানো এবং instruction সেট করা সম্ভব নয় এবং উভয় প্রসেসরগুলি মেমরি অবস্থানের পূর্ববর্তী মানটি পড়ে। সুতরাং, মূলত এই জিনিস সঠিক নয়। উভয় প্রসেসরের পক্ষে একই সাথে মেমরি অবস্থানের content পড়া এবং মেমরির মান নির্ধারণ করা সম্ভব নয়।

(স্লাইড সময় 07:56 দেখুন)

যাইহোক, প্রসেসর হার্ডওয়্যার দ্বারা আসলে কি নিশ্চিত হওয়া যায় যে যখন প্রসেসরগুলি প্রতিটি প্রসেসরের উপর চালিত হয় এবং উভয় প্রসেসর পরীক্ষা চালায় এবং ঠিক একই সময়ে instruction নির্দেশনা সেট করে তখন হার্ডওয়্যার নিশ্চিত করে যে এক প্রসেসর সমাপ্ত হবে পরবর্তী instruction এর আগে তার সম্পূর্ণ instruction টি চালাতে পারে। অতএব, এই ক্ষেত্রে একটি প্রক্রিয়া 0 এর মান পড়বে এবং অন্য প্রক্রিয়াটি 1 এর মান পড়বে। এখন আমরা দেখতে পাব কিভাবে এই বিশেষ instruction টি critical section এর সমস্যা সমাধান করতে ব্যবহৃত হয়।

(স্লাইড সময় 08:30 দেখুন)

এখানে এই বিশেষ snippet দেখায় কিভাবে একটি প্রক্রিয়া পরীক্ষাটি ব্যবহার করতে পারে এবং atomic instruction সেট করতে পারে যা critical section সমস্যার সমাধানের জন্য হার্ডওয়্যার সমর্থন করে। মূলত,

critical section লক করার জন্য আমাদের এখানে একটি while লুপ উপস্থিত থাকবে যা পরীক্ষাটি চালু করবে এবং একটি লক দিয়ে সেট করা হবে, যা মেমরি অবস্থানের সাথে আছে। এখন, এই যখন লুপ পরীক্ষা দ্বারা return value 0 সেট পর্যন্ত ক্রমাগত চালানো হবে; তাই যখন Test and Set নির্দেশ দ্বারা return value 0 হয় তারপর critical section প্রবেশ করা হয় এবং এই শেষে লক মান 0 সেট করা হয়। একাধিক প্রসেস যেমন একটি code snippet আছে, হার্ডওয়্যার গ্যারান্টি test and set একটি প্রক্রিয়ার জন্য একটি return value হবে 0। সব অন্যান্য প্রসেস এই while লুপে অবিরত ঘুরবে। তাই যখন এই প্রক্রিয়াটি critical section কার্যকর করে ফেলে, তখন লকটির মান 0 হবে। তাই লকটির মান 0 তে নির্ধারণ করা হলে পরীক্ষার return value থেকে 0 এর মান অর্জনের জন্য একটি প্রক্রিয়া হবে। অতএব, ঠিক অন্য একটি প্রক্রিয়া তারপর critical section প্রবেশ করবে। তাই Test and Set প্রথম আহ্বান একটি 0 পড়বে এবং লককে 1 এ সেট করে ফিরে আসবে। দ্বিতীয় Test and Set invocation 1 হিসাবে লকের মান দেখতে হবে এবং লুপ অবিরত চলবে লক 0 হওয়া অবধি। লক মান শুধুমাত্র 0 হয় যখন প্রথম প্রক্রিয়া এটি আনলক করে exclusively তে 0 সেট করে। সুতরাং এই পদ্ধতিতে আমরা দেখি যে হার্ডওয়্যার থেকে সামান্য সাহায্যের মাধ্যমে critical section সমস্যার সমাধান খুব কার্যকর।

(স্লাইড সময় 10:41 দেখুন)

Intel system Test and Set সমর্থন করে না। অন্যদিকে এটি exchange; xchg হিসাবে পরিচিত একটি instruction সমর্থন করে। সুতরাং একটি xchg instruction এখানে নেওয়া হয় এবং এটি একটি atomic instruction এবং xchg নির্দেশ দুটি parameter লাগে, একটি memory address এবং একটি integer নেয়। এখন কি করা হয় যে ঐ memory address এর content previous এর মধ্যে সংরক্ষিত করা হয়, তারপর v এর মান যা মান xchg এ পাস করা হয় তারপর যে মেমরি অবস্থান সংরক্ষিত হয় এবং previous মান return হয়। সুতরাং, মূলত এই কি অর্জন করে আমরা xchg register variable এর সাথে memory location বিনিময় করি। উদাহরণস্বরূপ, এই পদ্ধতিতে একটি প্রসেসরের চলমান প্রক্রিয়া দ্বারা xchg কে দেখবে, এটি একটি memory location এর সাথে একটি register value বিনিময় করবে এবং এই পুরো বিষয়টি atomic ভাবে সম্পন্ন করা হবে।

(স্লাইড সময় 11:49 দেখুন)

এমনকি যদি অন্য প্রক্রিয়া xchg একই সময়ে সঞ্চালিত হয় তবে Intel হার্ডওয়্যার নিশ্চিত করে যে এই দুটি অপারেশন আলাদাভাবে সম্পন্ন করা হয়। সুতরাং, প্রথম একটি প্রক্রিয়া xchg নির্দেশটি সম্পূর্ণ করবে করবে এবং কেবলমাত্র তখন দ্বিতীয় প্রক্রিয়ায় data বিনিময় চালু করবে।

(স্লাইড সময় 12:14 দেখুন)

সুতরাং এই বিশেষ স্লাইডটি দেখায় যে একটি Intel হার্ডওয়্যারের xchg নির্দেশ কীভাবে critical section সমস্যা সমাধান করতে ব্যবহৃত হয়। আমাদের Acquire এবং Release দুটি primitive function আছে। Acquire একটি critical section লক করার জন্য ব্যবহার করা হয়, একটি Release একটি critical section আনলক করতে ব্যবহার করা হয়। সুতরাং এই, Acquire এবং Release একটি Locked নামক memory location এ pointer হিসাবে পাস হয়। সুতরাং একটি লুপ বা একটি infinite অর্জনের পরিবর্তে, এই function xchg লক চালু হয়। সুতরাং, এখানে xchg উপস্থিত এবং এটি দুটি parameter প্রেরণ করা হয়; এই ক্ষেত্রে memory location টি লক এবং আপনি সেট করতে চান যে মান। তাই প্রথমে এই ক্ষেত্রে মান Intel প্রসেসরের eax রেজিস্টারে স্থানান্তরিত হয় এবং তারপর xchg নির্দেশ দেওয়া হয়। ফলস্বরূপ মেমরিতে সংরক্ষণ করা মেমরি বা ডাটা eax রেজিস্টারে লোড হয়ে যায়। এবং এই ক্ষেত্রে মান যা eax রেজিস্টারে লোড করা হয়েছিল মেমরিতে সংরক্ষিত হয়। এখন এই বিশেষ eax রেজিস্টার return করে। তাই এই লুপ ভেঙ্গে যাবে যদি xchg দ্বারা return value যে eax এর মান 0 হয়। লক release করার জন্য, আমরা কেবল লক মান 0 তে সেট করেছি। আসুন দেখি কিভাবে এটি কাজ করে। তাই প্রথমে, এখানে মেমরি অবস্থান 0 হবে এবং যখন প্রক্রিয়াটি xchg instruction নির্দেশ করে যে 0 মানটি eax রেজিস্টারে ঢুকিয়ে দেওয়া হয়, এবং 1 এর মান যা eax রেজিস্টারে উপস্থিত ছিল সেটি মেমরিতে আসে।

(স্লাইড সময় 14:29 দেখুন)

মেমরি 1 এর একটি মান আছে যখন প্রসেসরের উপস্থিত eax রেজিস্টারের একটি মান আছে 0 এবং এর ফলে লুপ থেকে প্রস্থান এবং critical section এ প্রবেশ করতে ব্যবহৃত হয়। এখন লক release করার জন্য, আমরা কেবল 0 এর লক মান সেট করে রেখেছি। এখন যখন একটি দ্বিতীয় প্রসেস চালু হয়, তখন এই লুপটি চলতে থাকবে যতক্ষণ না পর্যন্ত এই memory location মানটি প্রথমে 0 এ সেট করা হবে।

(স্লাইড সময় 15:08 দেখুন)

আমরা এখানে দেখি, লকটি মুক্ত করার জন্য আমরা কেবল মেমরির মান 0 তে সেট করি এবং তারপর অন্য প্রসেসরের মধ্যে সম্ভবত 0 এর মান অর্জন করতে সক্ষম হবে এবং critical section অর্জন করবে।

(স্লাইড সময় 15:23 দেখুন)

তাই xchg এর মতো এই higher level instruction গঠন করতে ব্যবহৃত হয় যা বিভিন্ন critical section সমস্যাগুলিতে ব্যবহৃত হয়। আমরা spinlock এবং mutex এই ভিডিওতে দেখব এবং পরবর্তী ভিডিওতে semaphore দেখব।

(স্লাইড সময় 15: 39 দেখুন)

স্পিনলক আগে আমরা দেখেছি, এর দুটি function আছে Acquire এবং Release। যদিও critical section access লাভের জন্য Acquire ব্যবহার করা হয় তবে এটি critical section লক করার জন্য ব্যবহৃত হয়, Release function critical section আনলক করতে ব্যবহৃত হয়। তাই এটা ব্যবহার করার জন্য দুটি প্রসেস থাকতে পারে; প্রক্রিয়া 1 এবং প্রক্রিয়া 2। Critical section এ প্রবেশ করার জন্য প্রক্রিয়াটির লকটি অর্জন করতে হবে এবং এই ফাংশনটি ক্রমাগতভাবে লুপে থাকবে যতক্ষণ না xchg দ্বারা 0 মান ফেরত আসে। তাই যখন xchg এর মান 0 হয় তখন এই লুপ ব্রেক করে এবং প্রক্রিয়া 1 critical section এ প্রবেশ করবে। লক release করার জন্য লক করা মানটি 0 দ্বারা সেট করা হয়। তাই, একটি প্রক্রিয়া একটি নির্দিষ্ট সময়ে লক অর্জন করবে এবং অন্য প্রসেসটি এই বিশেষ লুপে অপেক্ষা করবে এবং এটি ক্রমাগত xchg ফাংশন চালু করবে যতক্ষণ না এটি 0 হয়। সুতরাং লকটি মুক্ত করার জন্য প্রথম প্রক্রিয়াটি মানটিকে 0 তে সেট করে দেবে, এটি দ্বিতীয় প্রসেসটি এই লুপ থেকে বিরত করবে এবং critical section এ প্রবেশ করবে। spinlock.c এবং spinlock.h এর দুটি ফাইলের মাধ্যমে xv6 এ spinlock প্রয়োগ করার পদ্ধতি সম্পর্কে আরও বিস্তারিত আপনি দেখতে পারবেন।

(স্লাইড সময় 17:13 দেখুন)

এখন আমরা কিছু কিছু বিষয় দেখতে পাব যা xchg instruction ব্যবহার করে। তাই আমরা যেমন xchg instruction দেখেছি, তেমনি construct গুলির মধ্যে spinlock সবচেয়ে গুরুত্বপূর্ণ অংশ যেমন আমরা আগের স্লাইডে দেখেছি। সুতরাং, xchg নির্দেশের সাথে সম্পর্কিত সমস্যাগুলি কি বোঝা গুরুত্বপূর্ণ। মূলত xchg নির্দেশের এই বিশেষ বিন্যাস, একটি রেজিস্টার eax এবং একটি memory location X এর মধ্যে তথ্য বিনিময় ঘটে। এখন এটি নিশ্চিত করা উচিত যে এই variable X উপর কোন compiler optimization না হয়। সুতরাং, কিছু সাধারণ optimization এ X এর মান একটি রেজিস্টারের মধ্যে সংরক্ষিত করা সম্ভব। এই ক্ষেত্রে, আমরা কেবল একটি রেজিস্টার eax এবং অন্য একটি রেজিস্টারের মধ্যে তথ্য বিনিময় করছি যা উদ্দেশ্য সমাধান করবে না। অতএব, আমরা এই বিশেষ লুপটি লিখব assembly বা volatile keyword ব্যবহার করে। xchg নির্দেশ বাস্তবায়নের সময় আরেকটি প্রয়োজনীয়তা memory location গুলি পুনর্নির্মাণ করা হয় তা নিশ্চিত করতে হয়। CPU-র মেমরি লোড এবং স্টোরেজগুলি পুনর্বিন্যাস করা উচিত নয়। সুতরাং এটা অর্জন করার জন্য আমরা serialized instructions ব্যবহার করি, যা instruction ordering না করার জন্য বাধ্য করে। আমাদের জন্য

সৌভাগ্যক্রমে ডিফল্টভাবে xchg নির্দেশ ইতিমধ্যে সিরিয়লাইজেশন কার্যকর করে। তাই এই সম্পর্কে যত্ন নেওয়ার প্রয়োজন আমাদের কিছুই নেই।

(স্লাইড সময় 18:46 দেখুন)

যাইহোক, আমাদের আরও ঘনিষ্ঠভাবে cache memory এর অবস্থা দেখতে হবে। এখন, একটি তালিকা এবং একটি memory location এর মধ্যে xchg নির্দেশ তথ্য বিনিময় করে। এখন সিস্টেমে উপস্থিত প্রতিটি CPU তাদের নিজস্ব ব্যক্তিগত cache হতে পারে, উদাহরণস্বরূপ, CPU 0 এর নিজস্ব ব্যক্তিগত L1 cache রয়েছে; একইভাবে CPU 1 এর নিজস্ব L1 cache রয়েছে। তাই এটি নিশ্চিত করা উচিত যে এই X এর মান এই CPU-র প্রতিটি cache না যায়, মূলত CPU-র caching সম্ভব নয়। বরং, এই xchg নির্দেশের প্রতিটি সঞ্চালন অবশ্যই মেমরিতে যেতে হবে এবং X এর মান লোড বা সংরক্ষণ করতে হবে। দ্বিতীয়ত এটিও নিশ্চিত করা উচিত যখন একটি CPU এই X-এ পড়ে এবং লেখে, এটি অন্য কথায় হয় যখন একটি CPU তে এই সময় xchg নির্দেশ invoking হয় অন্য কোন CPU-এই এক্স এর কোনও পরিবর্তন করতে পারে না, এটি atomic প্রকৃতি ভাঙ্গা হবে। এটি করার জন্য CPU-কে একটি lock line হিসাবে চিহ্নিত করা হয় যা লক করা মেমোরি অ্যাকসেসের সিস্টেমের মধ্যে অন্য সব CPU গুলিকে জানাতে হয়। সুতরাং এই লক নির্দেশের ফলে, সমস্ত xchg নির্দেশ যা তথ্য পড়ে এবং লেখে তা একটি মেমরি থেকে পড়া এবং লেখার তথ্য হতে হবে। সুতরাং, এই performance hit অসাধারণ পরিমাণ হতে পারে।

(স্লাইড সময় 20:30 দেখুন)

তাই, একটি ভাল Acquire অর্জন করার জন্য, একটি Acquire যা আরও দক্ষ আমরা এই বিশেষ Acquire function এ একটি ছোট উন্নতি করব। তাই মূলত, আমরা এখানে একটি লুপ ব্যবহার করি এবং এই লুপের মধ্যে আমরা ক্রমাগত xchg ফাংশন আহ্বান করি এবং এটি 0 এর মান return করছে কিনা তা লক্ষ্য করি। তাই লক্ষ্য করুন, এই xchg instruction গুলির প্রত্যেকটির একটি বিশাল performance overhead রয়েছে কারণ এটির প্রয়োজন যে লক করা keyword এর X মান হল মেমরিতে যাবে এবং মেমরি থেকে তথ্য পড়তে এবং লিখতে পারবে। মূলত, cache সম্ভব নয় এবং সেইজন্য বিপুল overhead আছে। অন্যদিকে যদি আমরা এটিকে ছোটখাট পরিবর্তন করি তবে এখানে দেখানো Acquire function ভালভাবে উন্নতি করবে। তাই এখানে আমাদের দুটি loops আছে একটি নিয়মিত xchg লুপ যা আপনি জানেন একটি bus transaction ফলে এবং বিশাল overhead আছে, যখন একটি ভেতরের লুপ যা কেবল memory location লক মান check করে। তাই এই বিশেষ অভ্যন্তরীণ জিনিস cacheable হতে পারে এবং তাই, অনেক performance overhead হবে না, যখন, বহিরাগত যখন লুপ উল্লেখযোগ্য ওভারহেড থাকবে, কিন্তু এটি খুব প্রায়ই আহ্বান করা হয় না। বেশিরভাগ সময় এই বিশেষ cache memory পড়া হবে। এখন cache coherency protocol নিশ্চিত করবে যে যখন আরেকটি প্রক্রিয়া লক

মান পরিবর্তন করে, তখন এই প্রক্রিয়াটি লক করা পরিবর্তনের মানটি 1 থেকে 0 পর্যন্ত দেখতে পাবে এবং এটি while লুপ থেকে প্রস্থান করবে এবং এই বিশেষ বাইরের while loop এ ফিরে যাবে। xchg ফাংশনটি তখন লকড মান এবং register মান বিনিময় করে এবং মানটিকে 1 এ সেট করে দেয়, এবং সেইজন্য while লুপ থেকে প্রস্থান করে। ধন্যবাদ।